```
# STEP 1: IMPORT LIBRARIES
import pandas as pd
import numpy as np


# STEP 2: LOAD DATASET
# Upload bank_note_data.csv in Colab
from google.colab import files
uploaded = files.upload()

bnotes = pd.read_csv('BankNote_Authentication.csv')
print(bnotes.head(10))


# STEP 3: PREPARE DATA
x = bnotes.drop('class', axis=1)
y = bnotes['class']

print(x.head(2))
print(y.head(2))


# STEP 4: TRAIN TEST SPLIT (train_test ratio = 0.2)
from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2)


# STEP 5: MLP CLASSIFIER (activation function : relu)
from sklearn.neural_network import MLPClassifier

mlp = MLPClassifier(max_iter=500, activation='relu')
mlp.fit(x_train, y_train)

pred = mlp.predict(x_test)
print(pred)


# STEP 6: EVALUATION
from sklearn.metrics import classification_report, confusion_matrix

print(confusion_matrix(y_test, pred))
print(classification_report(y_test, pred))


# STEP 7: MLP CLASSIFIER (activation function : logistic)
mlp = MLPClassifier(max_iter=500, activation='logistic')
mlp.fit(x_train, y_train)

pred = mlp.predict(x_test)
print(pred)

print(confusion_matrix(y_test, pred))
print(classification_report(y_test, pred))


# STEP 8: MLP CLASSIFIER (activation function : tanh)
mlp = MLPClassifier(max_iter=500, activation='tanh')
mlp.fit(x_train, y_train)

pred = mlp.predict(x_test)
print(pred)

print(confusion_matrix(y_test, pred))
print(classification_report(y_test, pred))


# STEP 9: MLP CLASSIFIER (activation function : identity)
mlp = MLPClassifier(max_iter=500, activation='identity')
mlp.fit(x_train, y_train)

pred = mlp.predict(x_test)
print(pred)

print(confusion_matrix(y_test, pred))
```

```
print(classification_report(y_test, pred))


# STEP 10: TRAIN TEST SPLIT (train_test ratio = 0.3)
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.3)


# STEP 11: MLP CLASSIFIER (activation function : relu)
mlp = MLPClassifier(max_iter=500, activation='relu')
mlp.fit(x_train, y_train)

pred = mlp.predict(x_test)
print(pred)

print(confusion_matrix(y_test, pred))
print(classification_report(y_test, pred))


# STEP 12: MLP CLASSIFIER (activation function : logistic)
mlp = MLPClassifier(max_iter=500, activation='logistic')
mlp.fit(x_train, y_train)

pred = mlp.predict(x_test)
print(pred)

print(confusion_matrix(y_test, pred))
print(classification_report(y_test, pred))


# STEP 13: MLP CLASSIFIER (activation function : tanh)
mlp = MLPClassifier(max_iter=500, activation='tanh')
mlp.fit(x_train, y_train)

pred = mlp.predict(x_test)
print(pred)

print(confusion_matrix(y_test, pred))
print(classification_report(y_test, pred))


# STEP 14: MLP CLASSIFIER (activation function : identity)
mlp = MLPClassifier(max_iter=500, activation='identity')
mlp.fit(x_train, y_train)

pred = mlp.predict(x_test)
print(pred)

print(confusion_matrix(y_test, pred))
print(classification_report(y_test, pred))
```

```
Choose Files BankNote_…ntication.csv
BankNote_Authentication.csv(text/csv) - 46442 bytes, last modified: 4/11/2026 - 100% done
Saving BankNote_Authentication.csv to BankNote_Authentication (7).csv
   variance  skewness  curtosis  entropy  class
0   3.62160    8.6661  -2.80730 -0.44699      0
1   4.54590    8.1674  -2.45860 -1.46210      0
2   3.86600   -2.6383   1.92420  0.10645      0
3   3.45660    9.5228  -4.01120 -3.59440      0
4   0.32924   -4.4552   4.57180 -0.98880      0
5   4.36840    9.6718  -3.96060 -3.16250      0
6   3.59120    3.0129   0.72888  0.56421      0
7   2.09220   -6.8100   8.46360 -0.60216      0
8   3.20320    5.7588  -0.75345 -0.61251      0
9   1.53560    9.1772  -2.27180 -0.73535      0
   variance  skewness  curtosis  entropy
0    3.6216    8.6661   -2.8073 -0.44699
1    4.5459    8.1674   -2.4586 -1.46210
0    0
1    0
Name: class, dtype: int64
[0 0 1 1 1 1 1 1 0 0 0 0 0 1 0 1 1 0 0 0 0 0 1 1 1 0 0 0 1 0 0 0 1 0 0 1 0
 1 1 0 1 1 1 1 1 0 0 0 0 1 1 0 1 0 1 0 0 0 0 1 1 0 1 1 1 0 0 0 0 1 1 1 1 0
 0 1 0 0 1 0 0 0 1 1 0 0 0 1 1 1 1 0 1 1 0 0 1 1 0 1 1 0 0 1 0 0 1 1 1 0 1
 0 1 0 0 1 0 1 0 0 0 1 0 0 0 1 1 1 0 0 0 1 0 0 1 1 0 1 1 1 0 1 0 0 1 0 0 0
 0 1 1 1 0 1 0 0 0 0 0 1 0 1 0 0 1 1 1 1 0 1 0 0 1 1 0 0 1 0 0 1 1 0 0 1 0
 0 1 1 0 0 0 0 1 0 0 1 0 1 1 0 0 1 1 0 0 1 0 1 1 0 1 1 0 0 0 1 1 1 1 1 1 0
 1 0 1 1 0 1 1 0 0 0 1 0 1 0 1 1 0 0 0 1 0 0 1 0 0 1 1 0 1 0 0 1 0 1 1 0 1
 0 0 1 0 1 1 1 0 0 1 0 1 0 0 0 1]
[[145   0]
 [  0 130]]
              precision    recall  f1-score   support

           0       1.00      1.00      1.00       145
           1       1.00      1.00      1.00       130

    accuracy                           1.00       275
   macro avg       1.00      1.00      1.00       275
weighted avg       1.00      1.00      1.00       275

[0 0 1 1 1 1 1 1 0 0 0 0 0 1 0 1 1 0 0 0 0 0 1 1 1 0 0 0 1 0 0 0 1 0 0 1 0
 1 1 0 1 1 1 1 1 0 0 0 0 1 1 0 1 0 1 0 0 0 0 1 1 0 1 1 1 0 0 0 0 1 1 1 1 0
 0 1 0 0 1 0 0 0 1 1 0 0 0 1 1 1 1 0 1 1 0 0 1 1 0 1 1 0 0 1 0 0 1 1 1 0 1
 0 1 0 0 1 0 1 0 0 0 1 0 0 0 1 1 1 0 0 0 1 0 0 1 1 0 1 1 1 0 1 0 0 1 0 0 0
 0 1 1 1 0 1 0 0 0 0 0 1 0 1 0 0 1 1 1 1 0 1 0 0 1 1 0 0 1 0 0 1 1 0 0 1 0
 0 1 1 0 0 0 0 1 0 0 1 0 1 1 0 0 1 1 0 0 1 0 1 1 0 1 1 0 0 0 1 1 1 1 1 1 0
 1 0 1 1 0 1 1 0 0 0 1 0 1 0 1 1 0 0 0 1 0 0 1 0 0 1 1 0 1 0 0 1 0 1 1 0 1
 0 0 1 0 1 1 1 0 0 1 0 1 0 0 0 1]
[[145   0]
 [  0 130]]
              precision    recall  f1-score   support

           0       1.00      1.00      1.00       145
           1       1.00      1.00      1.00       130

    accuracy                           1.00       275
   macro avg       1.00      1.00      1.00       275
weighted avg       1.00      1.00      1.00       275

[0 0 1 1 1 1 1 1 0 0 0 0 0 1 0 1 1 0 0 0 0 0 1 1 1 0 0 0 1 0 0 0 1 0 0 1 0
 1 1 0 1 1 1 1 1 0 0 0 0 1 1 0 1 0 1 0 0 0 0 1 1 0 1 1 1 0 0 0 0 1 1 1 1 0
 0 1 0 0 1 0 0 0 1 1 0 0 0 1 1 1 1 0 1 1 0 0 1 1 0 1 1 0 0 1 0 0 1 1 1 0 1
 0 1 0 0 1 0 1 0 0 0 1 0 0 0 1 1 1 0 0 0 1 0 0 1 1 0 1 1 1 0 1 0 0 1 0 0 0
 0 1 1 1 0 1 0 0 0 0 0 1 0 1 0 0 1 1 1 1 0 1 0 0 1 1 0 0 1 0 0 1 1 0 0 1 0
 0 1 1 0 0 0 0 1 0 0 1 0 1 1 0 0 1 1 0 0 1 0 1 1 0 1 1 0 0 0 1 1 1 1 1 1 0
 1 0 1 1 0 1 1 0 0 0 1 0 1 0 1 1 0 0 0 1 0 0 1 0 0 1 1 0 1 0 0 1 0 1 1 0 1
 0 0 1 0 1 1 1 0 0 1 0 1 0 0 0 1]
```

```
 0 0 1 0 1 1 1 0 0 1 0 1 0 0 0 1]
[[143   2]
 [  1 129]]
              precision    recall  f1-score   support

           0       0.99      0.99      0.99       145
           1       0.98      0.99      0.99       130

    accuracy                           0.99       275
   macro avg       0.99      0.99      0.99       275
weighted avg       0.99      0.99      0.99       275

[1 1 0 1 0 0 0 0 0 1 0 1 0 0 0 0 0 0 0 0 1 0 1 0 1 1 0 0 1 0 0 1 0 0 1 1 0
 0 0 0 1 1 1 0 1 1 1 1 1 1 1 0 1 0 0 0 0 1 1 1 1 1 0 1 0 0 1 1 1 1 1 0 1 0
 0 0 0 1 0 1 0 1 1 1 0 0 1 0 0 1 1 1 1 0 1 0 1 0 1 1 0 0 0 1 1 0 1 0 1 0 1
 0 0 0 1 1 0 0 1 0 0 0 0 0 0 1 0 1 1 1 1 1 0 1 0 1 0 1 0 0 0 1 0 1 0 1 0 1
 0 0 0 0 0 1 0 1 0 1 0 1 1 1 1 1 1 0 0 1 0 1 0 0 0 1 1 0 1 0 1 0 0 1 1 0 0 1
 1 1 1 1 1 1 1 1 0 0 0 1 1 1 1 0 0 0 0 0 1 0 1 1 0 0 1 0 1 0 0 0 0 1 1 0 0
 0 0 1 0 0 1 0 0 0 1 1 0 0 0 0 1 0 0 1 1 0 0 1 0 1 0 1 0 0 1 0 0 0 1 1 1 1
 1 1 0 1 0 1 0 1 1 0 1 0 1 0 1 0 0 0 0 1 0 0 1 0 1 1 1 0 1 0 1 1 1 0 0 1 1
 0 1 1 0 1 1 1 1 0 0 0 1 0 1 0 0 0 0 0 0 0 1 1 1 0 0 1 1 0 0 1 1 1 0 0 1 0
 1 0 0 0 0 0 1 1 1 1 1 1 1 1 1 0 0 0 0 1 0 0 1 1 0 0 0 0 1 1 1 1 1 0 0 1 1
 1 0 0 0 0 1 0 1 0 0 0 1 1 0 0 1 0 0 0 1 0 0 1 0 1 1 1 0 0 1 1 0 0 1 0 0 0
 1 0 0 0 1]
[[217   0]
 [  0 195]]
              precision    recall  f1-score   support

           0       1.00      1.00      1.00       217
           1       1.00      1.00      1.00       195

    accuracy                           1.00       412
   macro avg       1.00      1.00      1.00       412
weighted avg       1.00      1.00      1.00       412

[1 1 0 1 0 0 0 0 0 1 0 1 0 0 0 0 0 0 0 0 1 0 1 0 1 1 0 0 1 0 0 1 0 0 1 1 0
 0 0 0 1 1 1 0 1 1 1 1 1 1 1 0 1 0 0 0 0 1 1 1 1 1 0 1 0 0 1 1 1 1 1 0 1 0
 0 0 0 1 0 1 0 1 1 1 0 0 1 0 0 1 1 1 1 0 1 0 1 0 1 1 0 0 0 1 1 0 1 0 1 0 1
 0 0 0 1 1 0 0 1 0 0 0 0 0 0 1 0 1 1 1 1 1 0 1 0 1 0 1 0 0 0 1 0 1 0 1 0 1
 0 0 0 0 0 1 0 1 0 1 0 1 1 1 1 1 1 0 0 1 0 1 0 0 0 1 1 0 1 0 1 0 0 1 1 0 0 1
 1 1 1 1 1 1 1 1 0 0 0 1 1 1 1 0 0 0 0 0 1 0 1 1 0 0 1 0 1 0 0 0 0 1 1 1 0
 0 0 1 0 0 1 0 0 0 1 1 0 0 0 0 1 0 0 1 1 0 0 1 0 1 0 1 0 0 1 0 0 0 1 1 1 1
 1 1 0 1 0 1 0 1 1 0 1 0 1 0 1 0 0 0 0 1 0 0 1 0 1 1 1 0 1 0 1 1 1 0 0 1 1
 0 1 1 0 1 1 1 1 0 0 0 1 0 1 0 0 0 0 0 0 0 1 1 1 0 0 1 1 0 0 1 1 1 0 0 1 0
 1 0 0 0 0 0 1 1 1 1 1 1 1 1 1 0 0 0 0 1 0 0 1 1 0 0 0 0 1 1 1 1 1 0 0 1 1
 1 0 0 0 0 1 0 1 0 0 0 1 1 0 0 1 0 0 0 1 0 0 1 0 1 1 1 0 0 1 1 0 0 1 0 0 0
 1 0 0 0 1]
```